



# Day 3: AWS Shared Responsibility Model & Identity Access Management (IAM)

Welcome back to ITNW 1009. Today we move from *understanding* the cloud to *securing* it – mastering the skills that make you an immediate asset to any employer running on AWS.

ITNW 1009 – DAY 3

AWS CLOUD PRACTITIONER ALIGNED

## Today's Agenda

# What We're Covering

In the next 75 minutes, we'll build the security foundation that employers look for in every cloud professional. Here's our roadmap:

01

---

### **AWS Shared Responsibility Model**

Draw the exact line between what AWS secures and what *you* secure — a question that appears on every cloud certification exam.

03

---

### **Policies, Least Privilege & MFA**

Write JSON permission documents using industry best practices and enforce multi-factor authentication to lock down your accounts.

02

---

### **IAM: Users, Groups & Roles**

Build the three foundational identity types that control who can access your cloud environment and what they can do once inside.

04

---

### **Real-World Lab & Case Study**

Apply everything hands-on with a live IAM configuration lab and analyze a real \$42,500 breach caused by a single misconfig.

## Learning Objectives

# By the End of This Session, You Will Be Able To...

### **Categorize Security Obligations**

Correctly classify any given cloud security task as either AWS's responsibility or the customer's responsibility under the Shared Responsibility Model.

### **Configure IAM Identities**

Create custom IAM Users, Groups, and Roles with attached JSON policy constraints directly inside the AWS Management Console.

### **Apply Least Privilege**

Design permission structures that restrict access to only what's required for a job function – the gold standard of enterprise security.

 These objectives map directly to the **AWS Certified Cloud Practitioner** exam domains. Every concept here is resume-ready and certification-aligned.

## Key Vocabulary

# Terms to Know for This Session

Master these eight terms and you'll speak the language of cloud security professionals. Bookmark this slide for your exam review.

### Shared Responsibility

The framework dividing security duties between AWS and the customer.

### IAM User

A permanent identity for one person or application with long-term credentials.

### IAM Group

A collection of users that share the same permission policies.

### IAM Role

A temporary identity assumed by services or users — no permanent credentials.

### Policy Document (JSON)

The structured rulebook defining exactly what an identity can and cannot do.

### MFA

Multi-Factor Authentication — a second proof of identity beyond a password.

### Least Privilege

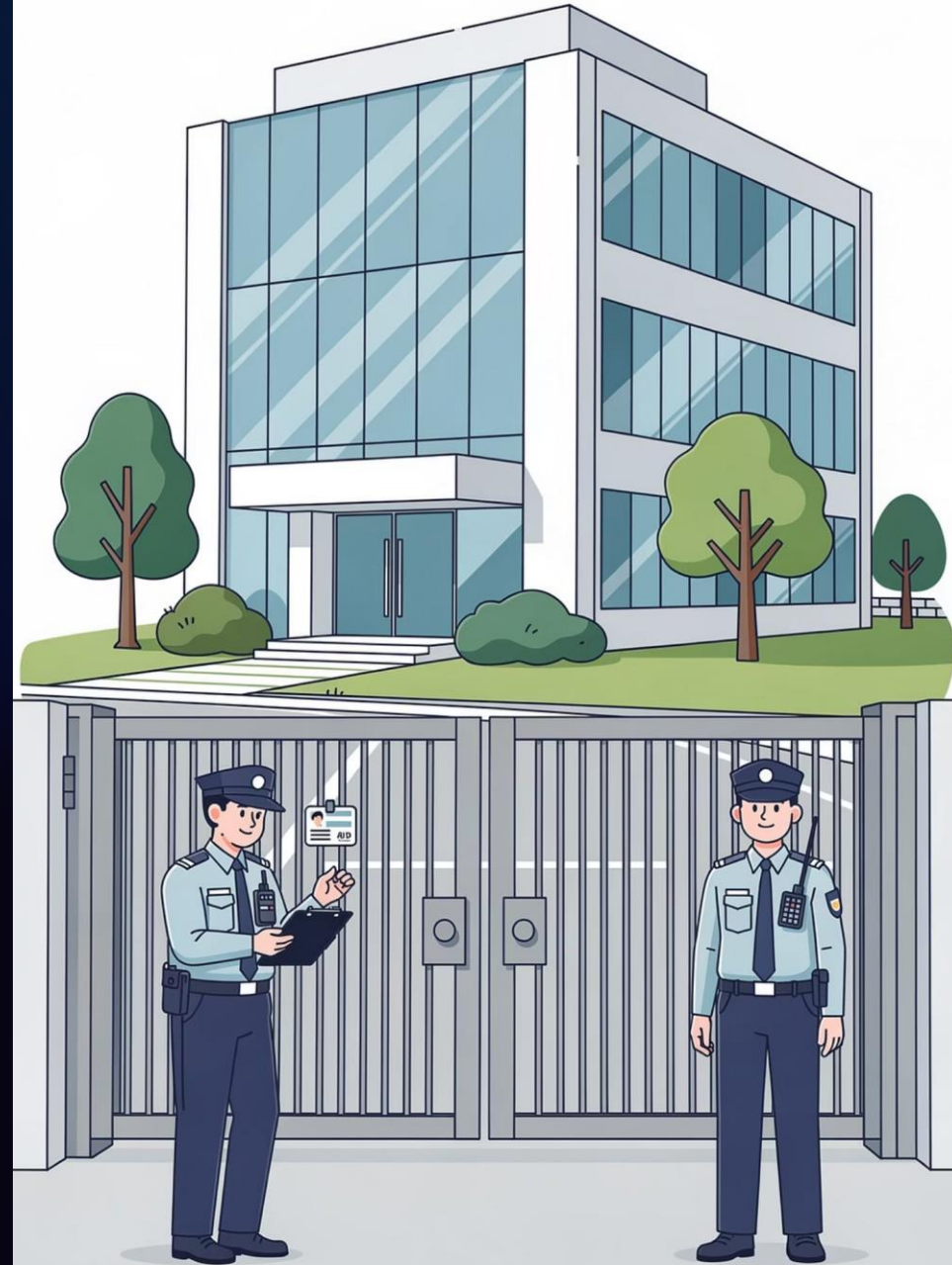
Only granting the minimum permissions required to complete a task.

### Principal

Any entity (user, role, service) that can make a request to AWS.

# Who Owns Security in the Cloud?

The answer isn't simple — it depends on where you draw the line. That boundary is defined by the **AWS Shared Responsibility Model**, and understanding it is the first thing every cloud professional must master.

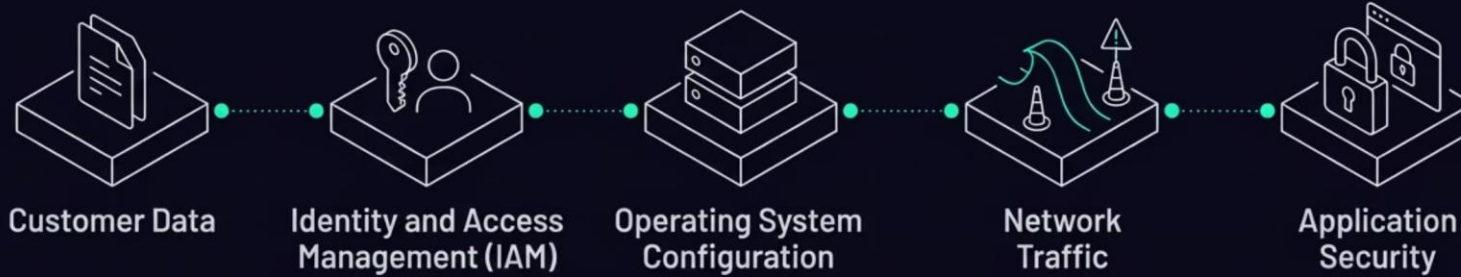




# The AWS Shared Responsibility Model

AWS divides all cloud security duties into two distinct zones. Knowing exactly which side of this line any task falls on is a core certification exam skill.

## Security IN the Cloud – Customer Responsibility



## Security OF the Cloud – AWS Responsibility



The dividing line is not fixed — it **shifts depending on the service type**. With IaaS (like EC2), customers manage more. With SaaS, AWS manages more. But the customer is *always* responsible for their data and IAM configurations.

## AWS Responsibility

# Security **OF** the Cloud — What AWS Owns

AWS invests billions of dollars annually to secure the physical and virtual infrastructure that powers the cloud. As a customer, you benefit from this infrastructure — but you never manage it directly.



### Physical Data Centers

Armed security guards, biometric entry, 24/7 surveillance, and strict visitor protocols protect every AWS facility worldwide.



### Hardware Maintenance

Failed hard drives, degraded networking equipment, and faulty power systems are all replaced and managed entirely by AWS engineers.



### Global Infrastructure

AWS maintains and secures its global network of Regions, Availability Zones, and Edge Locations — you simply consume the capacity.



### Host Virtualization

The underlying hypervisor software that runs your virtual machines is patched, updated, and secured entirely by AWS.

## Customer Responsibility

# Security **IN** the Cloud — What You Own

Once data, users, and applications live inside your AWS account, the responsibility is entirely yours. AWS cannot protect you from misconfiguration or poor access management decisions.



### Your Data

Data encryption at rest and in transit, classification, and backup strategies are entirely the customer's responsibility.



### Network Traffic Rules

Security groups, network ACLs, and VPC routing rules that control traffic flow must be configured and maintained by you.



### Identity & Access Management

Who can log in, what they can do, and how those credentials are protected are 100% customer-owned decisions.



### Operating Systems & Apps

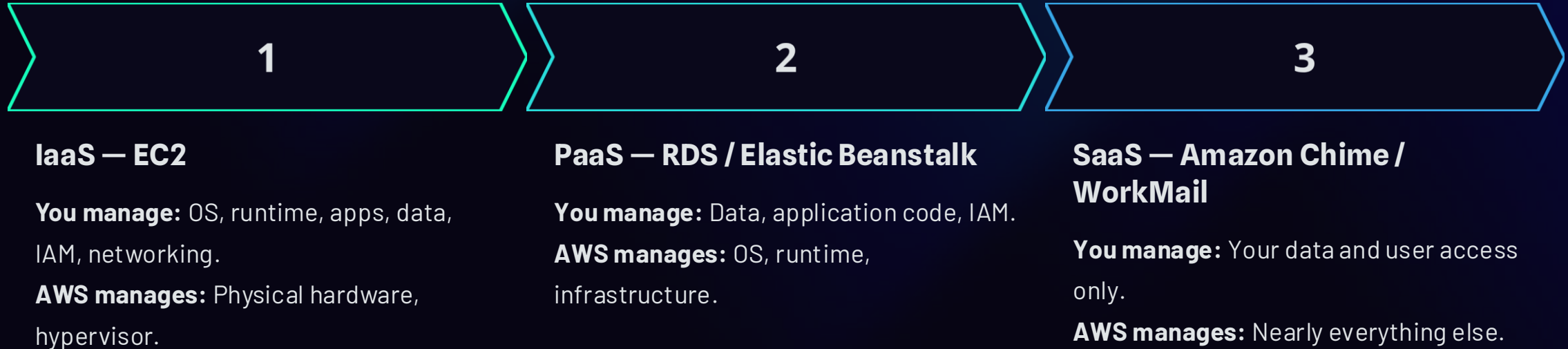
On IaaS services like EC2, OS patching, application updates, and configuration hardening fall on the customer's team.



## Responsibility Shifts by Service Type

# IaaS vs. PaaS vs. SaaS: The Line Moves

The deeper you go into managed services, the more AWS takes over. Understanding this shift helps you plan your security coverage correctly for every AWS service you deploy.



Quick rule: The more "managed" the service, the smaller your security surface — but you **always** own your data and IAM.



## Analogy: The Apartment Building

# Think of It Like Renting a High-Rise Apartment

### The Building Management Co. = AWS

The property management company secures the perimeter fencing, employs lobby security guards, maintains the master elevator system, monitors exterior cameras, and repairs the concrete foundation. **You never manage these systems.** You trust the building company to keep the facility standing and safe.

### The Tenant = You (The Cloud Customer)

The moment you walk through your apartment door, **responsibility shifts entirely to you.** If you leave your front door wide open, let untrustworthy guests in, or fail to lock the patio – the property management company cannot protect your belongings. You own the security inside your rented space.

## Analogy Continued

# Key Fobs, Access Tiers & the Lease Agreement

### **IAM Users → Individual Key Fobs**

Every family member gets their own uniquely coded electronic key fob. Each fob logs exactly who entered the building and at what time — individual accountability, just like IAM User credentials.

### **IAM Groups → Shared Amenity Tiers**

Instead of reprogramming every resident's fob individually, the building manager creates a *Premium-Amenity-Access* tier. Add a fob to the group and it instantly inherits pool, gym, and garage access.

### **IAM Policies → The Lease Agreement**

The signed rulebook states: **Effect:** **Allow** | **Action:** Access Fitness Center | **Resource:** Gym Floor | **Condition:** 5 AM–10 PM. Explicit, structured, and enforceable — just like a JSON policy document.

# Identity & Access Management

IAM is the gatekeeper of your entire AWS environment — controlling who gets in, what they can do, and for how long. Let's break it down.



# What Is IAM? The Big Picture

IAM (Identity and Access Management) is a **global AWS service** — you configure it once and it applies across all regions. It manages two critical phases every time someone tries to access your cloud resources.

## Authentication (AuthN)

*"Prove who you are."*

This is the front-door ID check. AWS verifies your identity through passwords, access keys, or MFA tokens before letting you anywhere near a resource. Like showing your state ID at a secure building entrance.

## Authorization (AuthZ)

*"Decide what you're allowed to do."*

Once authenticated, AWS checks your attached policies to determine which specific actions you can perform on which resources. Passing the front desk doesn't mean you can open every door.

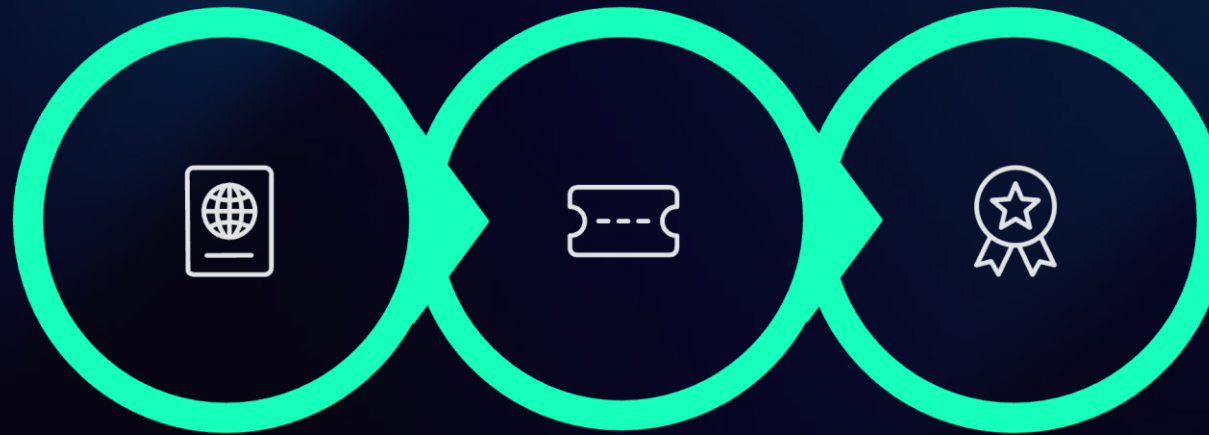


**Critical Default:** Every new IAM identity has **zero permissions** by default. Access must be explicitly granted. AWS denies everything unless a policy specifically allows it.



**IAM Analogy: The Airport**

# **Security Checkpoints, Boarding Passes & Flight Marshals**



**Passport**

**Boarding Pass**

**Flight Marshal**

The airport analogy maps perfectly onto IAM: your passport confirms identity (AuthN), your boarding pass defines permissions (AuthZ), and a deputized flight marshal represents a temporary Role assumption — credentials that expire the moment the flight lands.

## The Three Pillars of IAM Identity

# Users, Groups & Roles



### IAM User

Represents a **single person or application** requiring long-term credentials. Each user gets a unique username, login password, and/or API access keys. Use for human console logins and legacy app integrations.



### IAM Group

A **management collection** that applies the same permission policies to multiple users at once. Create groups like *Finance-Team* or *SysAdmins*, attach policies to the group, and users inherit everything automatically.



### IAM Role

An identity with **no permanent credentials**. Temporarily assumed by services (EC2, Lambda), applications, or cross-account users. Think of it as a hat an identity puts on for a task, then takes off when finished.

## IAM Users — Deep Dive

# When to Use an IAM User

### Best Use Cases

- Human administrators who need to log into the AWS Management Console
- Legacy applications that cannot use IAM Roles for authentication
- CI/CD pipelines in older architectures requiring programmatic access keys

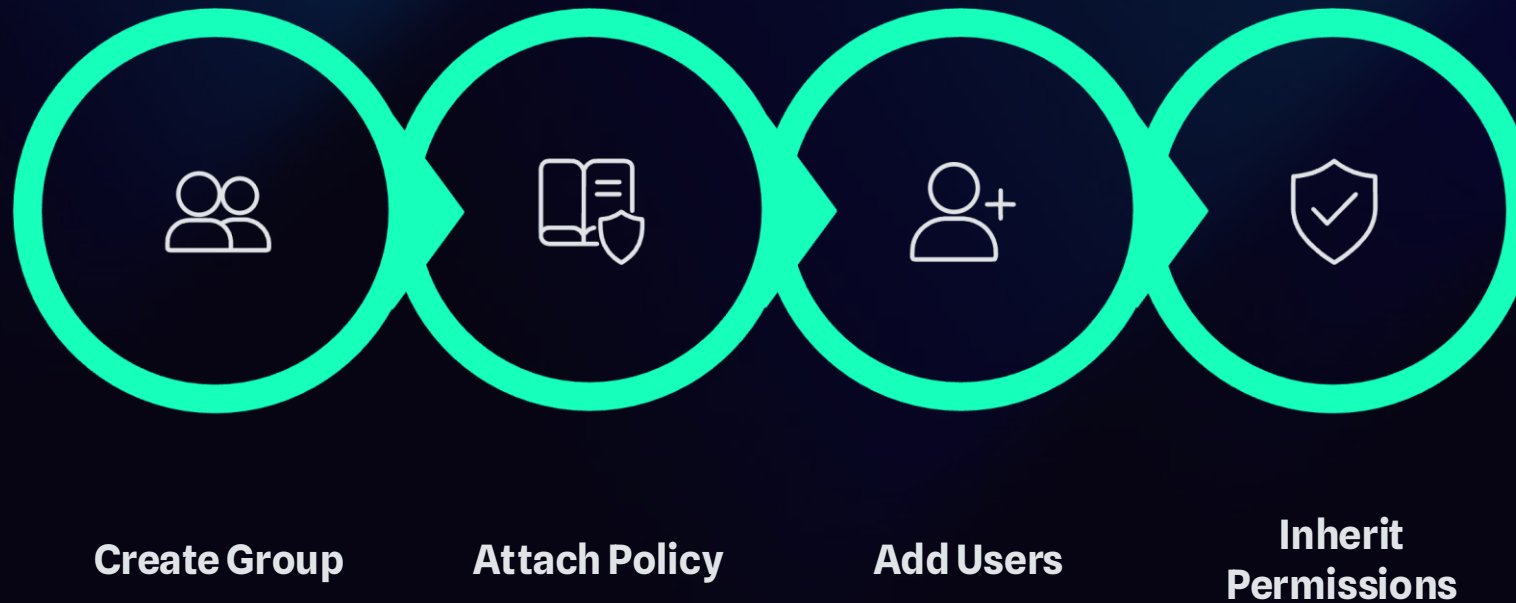
### Best Practices

- Always enable MFA for every human IAM User — no exceptions
- Never share credentials between individuals
- Rotate access keys every 90 days at minimum
- Disable or delete users immediately when they leave the organization



# Groups: Manage Permissions at Scale

In any real enterprise, manually assigning permissions to individual users is an unmanageable nightmare. Groups solve this completely — and they're the industry-standard approach.



When an employee transfers departments or leaves the company, you simply **remove them from the group** — their permissions are revoked instantly across all resources. No hunting through individual user policies.

# Roles: Temporary Credentials for Services & Apps

### What Makes Roles Different?

Roles have **no username or password**. Instead of embedding credentials in code, you attach a Role to an AWS service or user. AWS's Security Token Service (STS) issues *temporary* credentials that expire automatically — usually in 1–12 hours.

### Common Role Use Cases

- **EC2 → S3**: An EC2 web server reads files from S3 without any hardcoded keys
- **Lambda → DynamoDB**: A function writes records to a database securely
- **Cross-account access**: A contractor from another AWS account assumes a Role to do limited work
- **Federated login**: Corporate Active Directory users log in via SSO

📌 The flight marshal analogy: A pilot temporarily receives a high-security badge for one flight, then returns it upon landing. The Role expires — the pilot reverts to standard access.



The Golden Rule of Cloud Security

# Least Privilege.

Grant exactly what's needed to do the job. **Nothing more. Nothing less.** This single principle prevents the majority of cloud security breaches.

# The Principle of Least Privilege (PoLP)

The Principle of Least Privilege means you **never grant blanket administrative control** to an identity that only needs to read a single file or perform one specific task. You grant the minimum permissions required for the job — and nothing beyond that.

## What NOT to Do

Attach the `AdministratorAccess` managed policy to a developer because it's fast and convenient. This grants full, unrestricted access to every service in the account.

## What TO Do

Write a custom policy that allows only `s3:GetObject` on the specific bucket the developer needs to read. Nothing else is granted.



## Enforcing Least Privilege

# IAM Policy Documents: The Rulebook in JSON

Every permission in AWS is governed by a **Policy Document** — a structured JSON file that defines exactly what is allowed or denied, on which services, and for which specific resources.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "s3:GetObject",
        "s3:ListBucket"
      ],
      "Resource":
        "arn:aws:s3:::company-reports-bucket/*"
    }
  ]
}
```

## Breaking It Down

- **Effect:** Allow or Deny — the verdict on the request
- **Action:** The exact API calls permitted — `s3:GetObject` means "read a file"
- **Resource:** The specific AWS resource ARN — this limits scope to one bucket

If the action isn't listed here, AWS **denies it automatically**. No exceptions, no workarounds.

## Policy Anatomy

# Effect, Action & Resource — The Three Core Elements



### Effect

The verdict. Either `Allow` — the request is permitted — or `Deny` — the request is blocked. An explicit Deny always overrides any Allow, no matter what other policies say.



### Action

The specific AWS API call being evaluated. Examples: `s3:ListBucket`, `ec2:StopInstances`, `iam:CreateUser`. A wildcard `*` means all actions — use with extreme caution.



### Resource

The specific AWS resource ARN the policy applies to. You can target one S3 bucket, one EC2 instance, or use `*` for all resources. Always be as specific as possible.

❏ An IAM policy can also include a **Condition** block — for example, only allowing access between 9 AM–5 PM or only from a specific IP address range.

## Policy Types

# Managed Policies vs. Inline Policies

### AWS Managed Policies

Pre-built policies created and maintained by AWS. Examples: `AmazonS3ReadOnlyAccess`, `AdministratorAccess`. Great starting points, but often too broad for production use. Attach to users, groups, or roles.

### Inline Policies

Policies written directly into a single user, group, or role — not reusable. Good for one-off exceptions. Harder to audit and manage at scale, so use sparingly.

### Customer Managed Policies

Custom policies you write yourself, saved in your account and reusable across multiple identities. The recommended approach for any real enterprise environment. You control every Action and Resource.

### Permissions Boundaries

An advanced guardrail that sets the *maximum* permissions an identity can ever receive — even if someone tries to attach a broader policy later. Critical for delegated admin scenarios.



## MFA — Multi-Factor Authentication

# Your Password Alone Is Not Enough

Multi-Factor Authentication requires a second form of proof beyond just a password. Even if a bad actor steals credentials, they cannot access the account without the second factor. Enabling MFA on all privileged accounts is non-negotiable in any professional cloud environment.

### Virtual MFA

Apps like Google Authenticator or Authy generate a new 6-digit code every 30 seconds. Free, easy to set up, and accepted across all AWS services.

### Hardware MFA Token

A physical device (like a YubiKey) that generates TOTP codes. Preferred for root accounts and high-privilege admin users in regulated industries.

### MFA Enforcement Policy

You can write an IAM policy that *denies all actions* unless the user authenticated with MFA — forcing compliance automatically at the policy layer.

## MFA Enforcement

# Forcing MFA via IAM Policy

The best practice isn't just to *encourage* MFA — it's to **enforce** it programmatically. An MFA enforcement policy denies all actions until the user authenticates with their second factor.

```
{
  "Effect": "Deny",
  "NotAction": [
    "iam:CreateVirtualMFADevice",
    "iam:EnableMFADevice",
    "sts:GetSessionToken"
  ],
  "Resource": "*",
  "Condition": {
    "BoolIfExists": {
      "aws:MultiFactorAuthPresent": "false"
    }
  }
}
```

This policy says: *If MFA has NOT been used during this session, deny everything — except setting up MFA itself.* Attach this to a Group called `MFA-Required` and add all human users to it.

## IAM Best Practices Summary

# The Rules Every Cloud Professional Follows

### → Lock Down the Root Account

Enable MFA on the root account immediately after account creation. Never use root for day-to-day tasks — create an IAM admin user instead and lock root away.

### → Use Groups, Not Individual Policies

Assign permissions to Groups and add users to those groups. Never attach policies directly to individual users in production environments.

### → Prefer Roles Over Long-Term Keys

Wherever possible, use IAM Roles for applications and services. Long-term access keys stored in code or config files are one of the top causes of cloud breaches.

### → Audit Regularly with IAM Access Analyzer

Use AWS-native tools to continuously review which permissions are actively being used — and remove anything that hasn't been exercised in 90 days.

# Real-World Case Study

What happens when the rules get skipped? Let's look at a real incident that cost one company **\$42,500 in four hours**.



## Case Study

# The Costly Public GitHub Leak

**Cloud-Scale Solutions LLC** — a regional logistics subcontractor — hired a junior cloud engineer to build an automated data processing system integrated with AWS S3 storage.

### Late Night Development

Engineer creates an IAM User `Dev-Admin` with full `AdministratorAccess`. Generates long-term access keys and **hardcodes them** into a JavaScript config file.

1

### 2:03 AM — Keys Discovered

Automated malicious scanning bots crawling public repositories **find the AWS access key string within 3 minutes.**

2

3

### By 6:00 AM — \$42,500 Bill

150+ GPU-optimized EC2 instances spin up across 4 regions to mine cryptocurrency. Operations suspended for 24 hours during recovery.

4

5

### 2:00 AM — GitHub Push

Engineer pushes the entire codebase — including the credentials file — to a **public GitHub repository** to back up their work.

### 2:15 AM — Account Compromised

Attackers use the leaked keys to bypass all defenses, **delete legitimate admin profiles**, and lock the real team out of their own AWS console.

## Case Study — The Damage

# What Did This Breach Actually Cost?

**\$42,500**

### Unplanned AWS Bill

Racked up in under 4 hours from unauthorized cryptocurrency mining.

**150+**

### Unauthorized EC2 Instances

GPU-optimized instances launched across 4 global AWS regions.

**3 min**

### Time to Discovery by Bots

Automated scanning bots found the exposed key within 3 minutes of the GitHub push.

**24 hr**

### Operational Downtime

Regional business operations fully suspended during IAM infrastructure rebuild.



## Case Study — Root Cause Analysis

# What Went Wrong? Three Critical Failures

### Failure 1: Wrong Identity Type

The engineer created an **IAM User** with long-term access keys for a server task. An **IAM Role** attached directly to the EC2 instance should have been used instead — no keys required.

### Failure 2: No Least Privilege

Granting `AdministratorAccess` to an identity that only needed to interact with one S3 bucket. Only `s3:PutObject` and `s3:GetObject` on that specific bucket ARN should have been permitted.

### Failure 3: Credentials in Code

Hardcoding access keys into a plaintext application config file is never acceptable. Secrets must be managed through **AWS Secrets Manager** or **Systems Manager Parameter Store**, never in source code.

## Case Study — Shared Responsibility Analysis

# Who Is Responsible for This Breach?

### AWS's Role in This Incident

AWS successfully fulfilled every obligation under the Shared Responsibility Model:

- Physical data centers remained secure
- Host virtualization layers were uncompromised
- The global network infrastructure operated correctly
- AWS IAM enforced the permissions exactly as configured

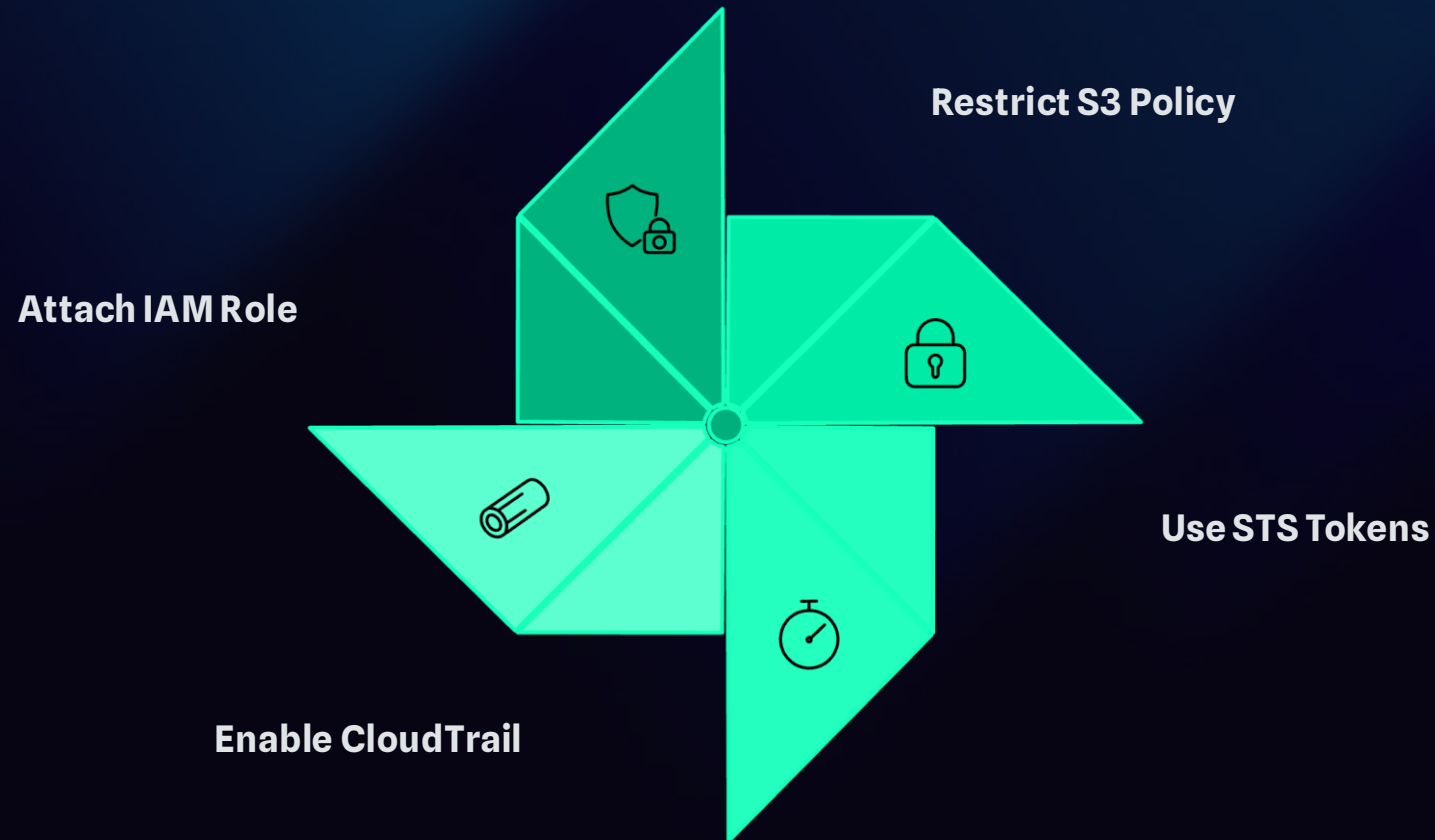
**AWS did nothing wrong.**

### Customer's Responsibility — 100% at Fault

Every failure was on the **customer side of the responsibility line**:

- IAM misconfiguration — wrong identity type used
- Excessive permissions — full admin instead of scoped access
- Credential management failure — keys in plaintext code
- No monitoring or budget alerts configured
- No service quotas to limit multi-region resource creation

# What Should Have Been Built Instead?



The entire breach was preventable with proper IAM architecture. IAM Roles eliminate the need for long-term credentials in application code entirely — temporary STS tokens are issued and expire automatically, leaving nothing to steal from a GitHub repository.

## Case Study — Proactive Defenses

# Corporate Guardrails to Prevent Future Incidents



### Git Secrets Scanners

Integrate tools like `git-secrets` or **Trufflehog** into your CI/CD pipeline. These tools automatically block any commit containing AWS credentials before the code reaches GitHub.



### AWS Budgets with Alerts

Configure AWS Budgets to send immediate email and SMS alerts when spend exceeds a threshold — even \$10 over baseline. Early detection stops runaway charges before they compound.



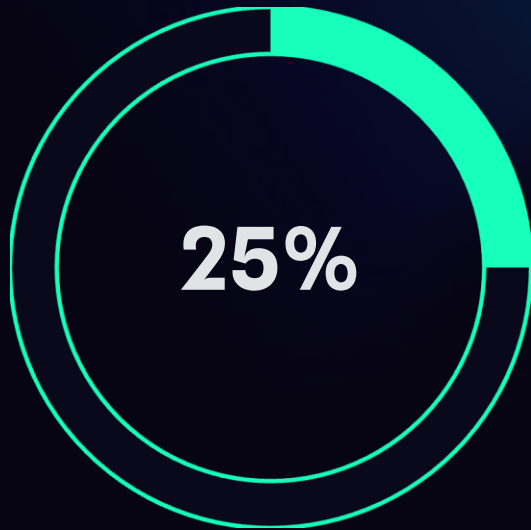
### Service Quotas & SCPs

Set hard EC2 launch limits per region using Service Quotas. Use AWS Organizations Service Control Policies (SCPs) to block resource creation in regions you never operate in.

## Certification Alignment

# How This Maps to the AWS CCP Exam

Every concept covered in this session appears directly in the **AWS Certified Cloud Practitioner** exam under the Security and Compliance domain — worth approximately 25% of your total score.



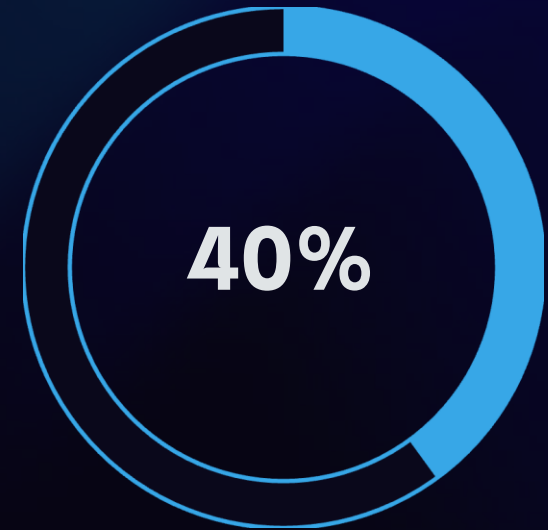
### Security Domain Weight

Security & Compliance is the second-largest domain on the CCP exam — and IAM is its cornerstone topic.



### Shared Responsibility Coverage

The Shared Responsibility Model appears in virtually every AWS certification exam at every level — from CCP to Solutions Architect Pro.

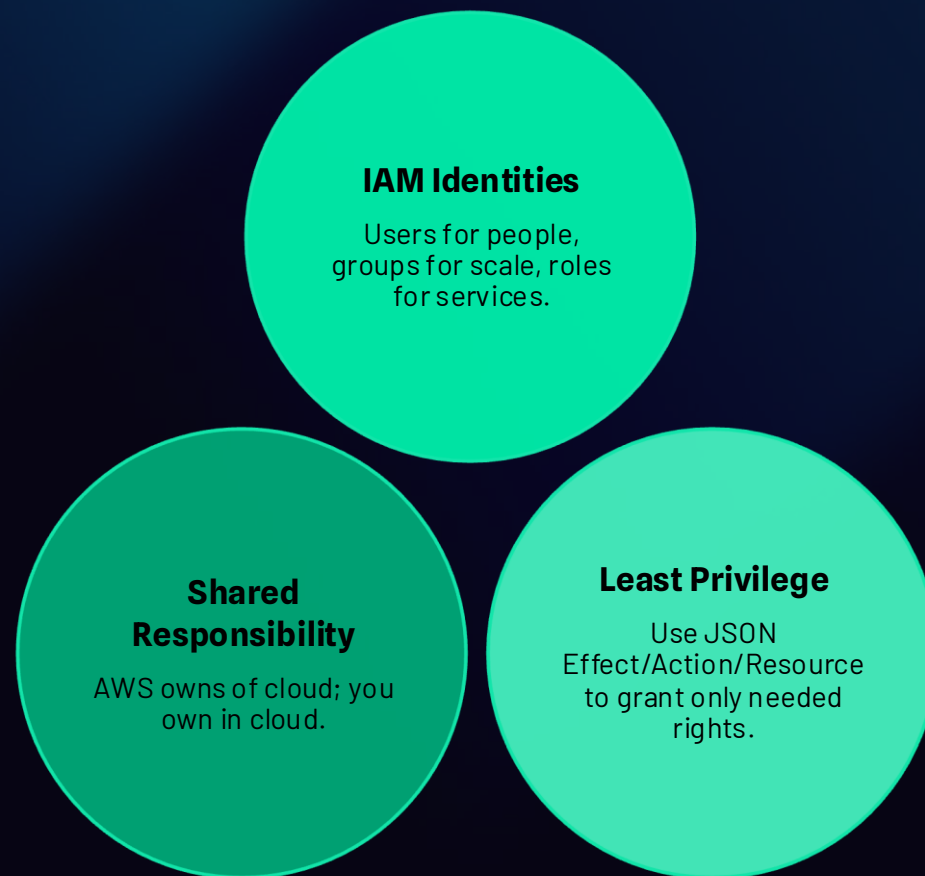


### IAM Question Frequency

Approximately 40% of Security domain questions on the CCP exam involve IAM concepts including users, roles, policies, and MFA enforcement.

## Day 3 — Key Takeaways

# What You Mastered



You now have the security vocabulary, conceptual framework, and hands-on IAM configuration skills that cloud employers hire for. These aren't just exam topics — these are the real controls that protect real businesses running on AWS every single day.



You've Got This

# Security is a skill. And you just built it.

You showed up, kept going, and learned the same security frameworks that AWS cloud engineers use every day at Fortune 500 companies.

That matters. **Keep showing up. Your career is being built right here.**